

SISTEM PENCARIAN RUTE BUS TRANS PADANG

Mata Kuliah: Struktur Data dan Algoritma

Program RuteBus

Tahun Akademik 2025/2026

Daftar Isi

1. Latar Belakang
2. Tujuan Program
3. Gambaran Umum Sistem
4. Struktur Program (File)
5. Data Koridor dan Halte
6. Struktur Data #1: Array of Struct
7. Struktur Data #2: Dynamic Array
8. Struktur Data #3: Binary Search Tree
9. Struktur Data #4: Queue (BFS)
10. Tipe Data Bentukan
11. Algoritma Pencarian Rute
12. Fitur Program
13. Alur Program
14. Contoh Eksekusi
15. Kesimpulan

1. Latar Belakang

Transportasi umum menjadi kebutuhan vital di kota-kota besar, termasuk Kota Padang. Bus Trans Padang hadir sebagai solusi transportasi massal yang melayani masyarakat melalui beberapa koridor rute.

Namun, masih banyak pengguna yang kesulitan mencari rute bus, terutama jika harus berpindah koridor (transit) untuk mencapai tujuan.

Oleh karena itu, dibuatlah program RuteBus untuk membantu pengguna menemukan rute dari suatu halte ke halte lain secara otomatis, dengan memanfaatkan konsep-konsep struktur data dan algoritma yang telah dipelajari.

2. Tujuan Program

- Menyediakan sistem pencarian rute bus Trans Padang secara otomatis
- Mengelola data halte dan koridor (tambah, hapus, simpan)
- Menerapkan struktur data: Array of Struct, Dynamic Array, Tree, dan Queue
- Menerapkan algoritma: Breadth-First Search (BFS) dan Backtracking
- Menampilkan rute lengkap dengan informasi transit antar koridor

Ruang Lingkup

- 3 koridor bus Trans Padang dengan total 53 halte
- Bahasa pemrograman C
- Data disimpan dalam file eksternal (halte.txt)
- Berbasis terminal (CLI)

3. Gambaran Umum Sistem

Program RuteBus adalah aplikasi CLI (Command Line Interface) yang memungkinkan pengguna mencari rute bus Trans Padang dari halte asal ke halte tujuan. Program ini dibangun dengan bahasa C dan mengimplementasikan 4 struktur data utama serta algoritma BFS dan Backtracking.

Ilustrasi Sistem

[INPUT] User memasukkan nama halte asal dan tujuan

|

v

[PROSES] Validasi -> Bangun Tree -> BFS -> Backtrack

|

v

[OUTPUT] Rute lengkap dengan titik transit

Pengguna Program

- Mahasiswa/masyarakat yang ingin mencari rute bus Trans Padang
- Operator system yang ingin mengelola data halte

4. Struktur Program (File)

Program terdiri dari 4 file source code, 1 file data, dan 1 file dokumentasi:

File	Baris	Fungsi
main.c	142	Entry point program, menu utama, input user
connection.h	108	Header: deklarasi struct Corridor dan fungsi
connection.c	704	Implementasi utama: load data, cari rute, CRUD halte
BinSTree.h	74	Header: deklarasi struct TreeNode dan operasi tree
BinSTree.c	214	Implementasi node tree, traversal, serialisasi
halte.txt	56	File data koridor dan halte
struktur-data.md	222	Dokumentasi struktur data

Total source code: ~1.060 baris, 25+ fungsi

5. Data Koridor dan Halte

Program mengelola 3 koridor bus Trans Padang dengan total 53 halte:

Koridor	Jumlah	Rute Utama
Koridor I	17	Pool Damri -> RTH Imam Bonjol -> Masjid Raya -> Basko GM -> UNP -> Terminal Anak Air
Koridor II	15	RTH Imam Bonjol -> Pasar Gaung -> Pantai Nirwana -> Bungus -> Simpang Sg Pisang
Koridor III	21	Terminal Anak Air -> Univ Terbuka -> RS Semen Padang -> Simpang Lubeg -> Lantamal

Titik Transit (Halte Penghubung Koridor)

RTH Imam Bonjol Padang

- Menghubungkan Koridor I dengan Koridor II

Terminal Anak Air

- Menghubungkan Koridor I dengan Koridor III

6. Struktur Data #1: Array of Struct

Array of Struct adalah struktur data yang menggabungkan array (kumpulan data dengan tipe sama) dan struct (kumpulan data dengan tipe berbeda dalam satu kesatuan).

Implementasi pada Program

```
typedef struct {
    char name[50];          // Nama koridor
    char** halte;           // Dynamic array of strings
    int jumlah_halte;       // Jumlah halte terisi
    int kapasitas;          // Kapasitas teralokasi
} Corridor;

Corridor corridors[MAX_CORRIDORS]; // 10 koridor
int jumlah_koridor;
```

Peran

- Menyimpan seluruh data master di memori selama program berjalan
- Menjembatani data dari file ke dalam program (load_data)
- Menyediakan data untuk ditampilkan ke user
- Menampung perubahan data dari proses tambah dan hapus

Analisis

- Input: File eksternal dengan format #NamaKoridor diikuti daftar halte
- Proses: Data dari file dibaca baris per baris, disimpan ke array of struct
- Output: Data ditampilkan ke layar dan ditulis kembali ke file

7. Struktur Data #2: Dynamic Array

Dynamic array adalah array yang ukurannya dapat berubah secara otomatis saat program berjalan. Berbeda dengan array statis, dynamic array dapat tumbuh (grow) saat item ditambah dan mengecil (shrink) saat item dihapus.

Implementasi pada Program

```
// Di dalam struct Corridor:
char** halte;      // Pointer ke array of pointers
int jumlah_halte;  // Jumlah terpakai
int kapasitas;     // Kapasitas teralokasi

// Strategi Grow: double kapasitas jika penuh
// Strategi Shrink: setengah jika <25% terpakai
```

Peran

- Menyimpan daftar halte dalam setiap koridor secara fleksibel
- Menghilangkan batas maksimal jumlah halte per koridor
- Mengoptimalkan penggunaan memori

Fungsi terkait

- pastikan_kapasitas() -> grow otomatis
- hapus_halte() -> shrink otomatis
- load_data() -> alokasi sesuai data file
- tambah_halte() -> grow jika perlu

8. Struktur Data #3: Binary Search Tree

Berbeda dengan BST konvensional, program ini menggunakan sequential chain (right-skewed tree) di mana `rightChild` menunjuk ke node berikutnya dalam urutan rute, bukan berdasarkan perbandingan ID.

Implementasi pada Program

```
typedef struct TreeNode {
    int id;                // koridor*100 + index
    char name[100];        // Nama halte
    int jalur;             // 0=biasa, 1=transit
    struct TreeNode* leftChild; // Selalu NULL
    struct TreeNode* rightChild; // Node berikutnya (chain)
    struct TreeNode* parent;    // Node sebelumnya
} TreeNode;
```

Struktur Sequential Chain

Asal -> rightChild -> Halte2 -> rightChild -> ... -> rightChild -> Tujuan

^

^

|-- parent pointer untuk backtrack --|

Peran

- Representasi sementara dari rute (dibuat saat pencarian, dihancurkan setelahnya)
- parent pointer memudahkan backtrack dari tujuan ke asal
- Bersifat sementara -> `destroyTree()` setelah selesai

9. Struktur Data #4: Queue (BFS)

Queue adalah struktur data yang menerapkan prinsip FIFO (First In, First Out). Elemen yang pertama masuk akan menjadi yang pertama keluar. Queue digunakan untuk mendukung algoritma BFS dalam pencarian node tujuan.

Implementasi pada Program

```
#define MAX_Q 500

static TreeNode* bfs_cari(TreeNode *root, const char *target) {
    static TreeNode *q[MAX_Q]; // Queue array statis
    int f = 0, r = 0;           // front dan rear
    q[r++] = root;              // Enqueue root

    while (f < r) {
        TreeNode *cur = q[f++]; // Dequeue
        if (match) return cur;   // Ditemukan
        if (left) q[r++] = left;
        if (right) q[r++] = right;
    }
    return NULL;
}
```

Peran

- Mendukung proses pencarian pada tree dengan metode BFS
- Menyimpan node-node yang akan dikunjungi secara berurutan
- Bersifat sementara - hanya aktif selama pencarian

Ringkasan 4 Struktur Data

Struktur Data	Fungsi	Input	Output
Array of Struct	Penyimpanan data master koridor	File eksternal	Layar + file
Dynamic Array	Daftar halte per koridor	Keyboard (user)	Layar + file
Binary Search Tree	Representasi rute sementara	Array of struct	Pohon di memori
Queue	Pemrosesan BFS	Node tree	Node tujuan

10. Tipe Data Bentukan

Tipe Data Primitif

Tipe	Contoh Variabel	Kegunaan
int	id, jumlah_halte, kapasitas, pilihan	Bilangan bulat: ID, indeks, jumlah
char	buf[10]	Input menu dan konfirmasi
char[]	name[50], name[100], asal[100], tujuan[100]	String: nama koridor, nama halte

Tipe Data Kolektif

Jenis	Cara Deklarasi	Kegunaan
Array of Struct	Corridor corridors [MAX_CORRIDORS]	Menampung 10 koridor
Dynamic Array	char** halte (via realloc)	Menampung nama halte per koridor
Array of Pointers (Stack)	TreeNode* stack_rute [MAX_STACK]	Stack untuk backtracking
Array of Pointers (Queue)	TreeNode* q[MAX_Q]	Queue untuk BFS

11. Algoritma Pencarian Rute

Saat user memilih "Cari Rute", program menjalankan rangkaian algoritma berikut:

Langkah-langkah

Langkah 1: Validasi

- Cari nama halte asal dan tujuan di semua koridor via `find_stop()`
- Jika tidak ditemukan, tampilkan error dan keluar

Langkah 2: Bangun Sequential Chain Tree

- Root = node asal
- Insert bentang dari asal menuju titik transit (jika beda koridor)
- Insert bentang dari transit menuju tujuan
- 4 kemungkinan kasus: sama koridor, asal utama, tujuan utama, beda cabang
- Penanganan arah: maju (dari < ke) atau mundur (dari > ke)

Langkah 3: BFS (Breadth-First Search)

- Cari node tujuan dalam tree menggunakan queue
- Kunjungi node level demi level (walaupun chain, BFS tetap jalan)
- Return node jika ditemukan

Langkah 4: Backtracking

- Mulai dari node tujuan, naik via parent pointer
- Push setiap node ke `stack_rute[]`
- Node transit (jalur==1) juga di-push ke `stack_transit[]`

Langkah 5: Cetak Rute

- Pop `stack_rute` dari atas (asal) ke bawah (tujuan)
- Cetak nama halte + info koridor
- Jika koridor berubah -> cetak "-- TRANSIT --"

Langkah 6: Cleanup

- Simpan tree ke file `tree.txt` (serialisasi)
- Hapus semua node dari memori via `destroyTree()`

12. Fitur Program

Program memiliki 4 menu utama dengan fitur-fitur berikut:

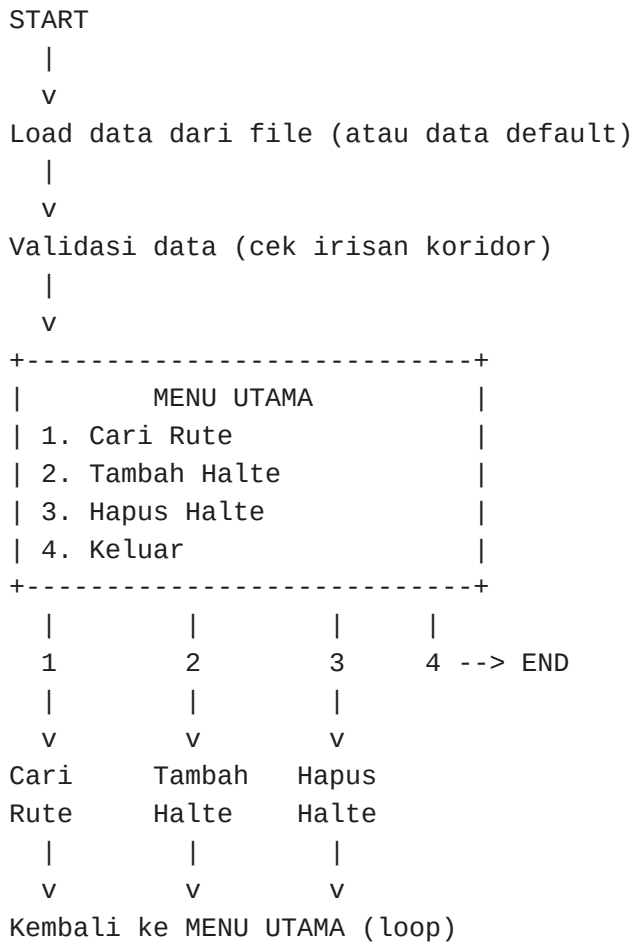
Fitur	Deskripsi	Proses
Cari Rute	Cari rute antar 2 halte	Validasi -> build tree -> BFS -> Backtrack -> Cetak -> Simpan
Tambah Halte	Tambah halte baru ke koridor	Pilih koridor -> Input nama -> Validasi duplikat -> Simpan file
Hapus Halte	Hapus halte dari koridor	Pilih koridor -> Lihat daftar -> Pilih nomor -> Konfirmasi -> Simpan
Keluar	Keluar dari program	return 0

Fitur Tambahan

- Validasi data otomatis saat startup (cek irisan koridor)
- Konfirmasi sebelum menghapus halte
- Validasi duplikat saat menambah halte
- File persistensi (data tersimpan setelah program ditutup)
- Data default jika file tidak ditemukan

13. Alur Program

Diagram alur program dari start hingga exit:



Flowchart: Cari Rute

```
CARI RUTE
|
v
Input: halte asal & tujuan
|
v
Cari halte di database
|
+-- Tidak ditemukan --> Error: "Halte tidak ditemukan"
|
v (ditemukan)
Tentukan kasus rute:
- Sama koridor?
- Asal di koridor utama?
- Tujuan di koridor utama?
- Keduanya di koridor cabang?
|
v
Bangun Sequential Chain Tree
(insert_bentang per segmen)
|
v
BFS: cari node tujuan dalam tree
|
+-- Tidak ditemukan --> Error
|
v (ditemukan)
Backtrack: dari tujuan ke asal
(via parent pointer, push ke stack)
|
v
Cetak rute dari stack
(deteksi transit antar koridor)
|
v
Simpan tree ke file + destroyTree
|
v
Kembali ke MENU
```

14. Contoh Eksekusi

Contoh 1: Rute dalam satu koridor

```
Masukkan nama halte asal   : Basko Grand Mall
Masukkan nama halte tujuan : Universitas Negeri Padang Air Tawar

=====
RUTE: Basko Grand Mall  -->  Universitas Negeri Padang Air Tawar
=====
Basko Grand Mall (Koridor 1)
Simpang Labor
Universitas Negeri Padang Air Tawar (tujuan)
=====
```

Contoh 2: Rute dengan transit

```
Masukkan nama halte asal   : Basko Grand Mall
Masukkan nama halte tujuan : Pasar Gaung

=====
RUTE: Basko Grand Mall  -->  Pasar Gaung
=====
Basko Grand Mall (Koridor 1)
Simpang Labor
RTH Imam Bonjol Padang
-- TRANSIT: RTH Imam Bonjol Padang (Koridor 1 --> Koridor 2) --
Masjid Nurul Iman
Pasar Gaung (tujuan)
=====
```

Contoh 3: Tambah Halte

```
=== DAFTAR KORIDOR ===
1. Koridor I (17 halte)
2. Koridor II (15 halte)
3. Koridor III (21 halte)
=====
Pilih koridor (1-3): 1
Masukkan nama halte baru: Halte Baru UNP
Halte 'Halte Baru UNP' berhasil ditambahkan ke Koridor I.
```

Contoh 4: Hapus Halte

```
=== DAFTAR KORIDOR ===
1. Koridor I (17 halte)
2. Koridor II (15 halte)
3. Koridor III (21 halte)
=====
Pilih koridor (1-3): 2

=== DAFTAR HALTE - Koridor II ===
1. RTH Imam Bonjol Padang
2. Masjid Nurul Iman
3. Pasar Gaung
...
=====
Pilih nomor halte (1-15): 3
Hapus 'Pasar Gaung' dari Koridor II? (y/n): y
Halte berhasil dihapus.
```

15. Kesimpulan

Program RuteBus berhasil mengimplementasikan konsep-konsep struktur data dan algoritma dalam sebuah aplikasi nyata pencarian rute bus Trans Padang.

Pencapaian

- Berhasil menerapkan 4 jenis struktur data (Array of Struct, Dynamic Array, Binary Search Tree, Queue)
- Berhasil menerapkan algoritma BFS dan Backtracking dalam pencarian rute
- Sistem dapat menangani rute dalam satu koridor maupun lintas koridor dengan transit
- Data tersimpan secara persistendalam file eksternal
- Fitur CRUD (Create, Read, Delete) untuk manajemen data halte

Struktur Data yang Digunakan

Struktur Data	Implementasi	Kegunaan
Array of Struct	Menyimpan data koridor dan halte di memori	Penyimpanan data master
Dynamic Array	Alokasi fleksibel untuk daftar halte	Grow/shrink otomatis
Binary Search Tree	Sequential chain untuk representasi rute	Backtrack via parent pointer
Queue	FIFO untuk BFS	Pencarian node tujuan

SEKIAN

Terima Kasih

Sistem Pencarian Rute Bus Trans Padang - RuteBus

Mata Kuliah: Struktur Data dan Algoritma